

ORGNZ-03 LAB: Bucket Strategy and Design - Hands-on Exercises

Series: ORGNZ — Organize Data: Buckets, Segments, Security | **Notebook:** 3 of 10 |
Type: LAB | **Created:** February 2026 | **Last Updated:** 04/25/2026

Overview

This lab notebook contains 3 hands-on exercises extracted from **ORGNZ-03: Bucket Strategy and Design**. Complete the lecture notebook first, then work through these exercises to reinforce the concepts with real DQL queries against your Dynatrace environment.

Table of Contents

- [Exercise 1: Recommended Naming Patterns](#)
- [Exercise 2: Recommended Naming Patterns](#)
- [Exercise 3: Recommended Naming Patterns](#)
- [Lab Summary](#)

Prerequisites

Requirement	Details
Completed	ORGNZ-03: Bucket Strategy and Design (lecture notebook)
Dynatrace Environment	SaaS tenant with Grail enabled
Permissions	<code>logs.read</code> , <code>metrics.read</code> , <code>entities.read</code> , <code>spans.read</code>

Exercise 1: Recommended Naming Patterns

ORGNZ-03: Bucket Strategy and Design

Series: ORGNZ — Organize Data: Buckets, Segments, Security | **Notebook:** 3 of 10 |
Created: January 2026 | **Last Updated:** 04/25/2026

A well-defined bucket strategy optimizes query performance, controls costs, and ensures compliance. This notebook covers naming conventions, retention planning, organizational alignment, and common bucket design patterns.

| Requirement | Details |

|---

-
1. Bucket Naming Conventions
 2. Retention Strategy
 3. Bucket Design Patterns
 4. Cost Control and Attribution
 5. Analyzing Bucket Usage
 6. Bucket Strategy Considerations
 7. Routing Data to Buckets
 8. Bucket Design Checklist
-

-----|-----|

| **Dynatrace Environment** | SaaS environment with Grail enabled |

| **Permissions** | `storage:bucket-definitions:write` for bucket creation |

| **Knowledge** | Completed ORGNZ-02 |

By the end of this notebook, you will:

- Design effective bucket naming conventions
- Plan retention strategies for different use cases
- Understand cost implications of bucket design
- Apply organizational alignment patterns

Pattern	Format	Example	Use Case
Provider-based	<provider>_<type>_<retention>	aws_logs_35d	Multi-cloud environments
LOB-based	<provider>_<type>_<lob>	aws_logs_finance	Line of business cost attribution
Team-based	team_<name>_<type>	team_platform_logs	Team ownership and billing
Compliance	<regulation>_<type>_<retention>	hipaa_logs_7y	Regulatory requirements

Bucket Name	Provider	Data Type	Retention	Org Unit
aws_logs_35d_platform	AWS	logs	35 days	Platform team
azure_metrics_90d_finance	Azure	metrics	90 days	Finance
gcp_spans_14d_checkout	GCP	spans	14 days	Checkout service
audit_logs_365d_compliance	Any	logs	365 days	Compliance
debug_logs_7d	Any	logs	7 days	Development

Rule	Requirement
First character	Must be a letter
Allowed characters	Lowercase alphanumeric, underscores, hyphens
Case	Lowercase only
Reserved names	Cannot use default_* prefix
Immutability	Names cannot be changed after creation

Use Case	Recommended Retention	Rationale
Debug/Development	3-7 days	Short-term troubleshooting, high volume
Standard Operations	35 days	Monthly trends, incident response

Use Case	Recommended Retention	Rationale
Performance Analysis	60-90 days	Quarterly comparisons
Compliance/Audit	365-3657 days	Regulatory requirements

Data Type	Typical Retention	Notes
Debug logs	3-7 days	High volume, low long-term value
Application logs	35-90 days	Operational analysis
Audit logs	1-7 years	Compliance requirements
Metrics	35-90 days	Trending and alerting
Spans	10-35 days	APM troubleshooting
Security events	90-365 days	Security investigations

Store critical audit information for extended periods:

```

Bucket: compliance_audit_logs
Table: logs
Retention: 1825 days (5 years)
Use case: SOX compliance, security audits
Route via: OpenPipeline filter on audit-level logs

```

Reduce costs by storing verbose logs briefly:

```

Bucket: debug_logs_7d
Table: logs
Retention: 7 days
Use case: Development troubleshooting
Route via: OpenPipeline filter on loglevel=DEBUG

```

Separate buckets for team-based access and cost attribution:

```

Bucket: team_platform_logs
Table: logs
Retention: 35 days
Use case: Platform team owns infrastructure logs
Access: IAM policy restricts to platform team

```

Business units with specific retention needs:

Bucket: finance_app_logs
Table: logs
Retention: 180 days
Use case: Financial application compliance
Cost: Chargeback to Finance cost center

Factor	Impact on Cost
Ingest volume (GiB/day)	Direct relationship
Retention period	Storage costs increase with longer retention
Query frequency	Query costs accumulate

Organization Level	Bucket Strategy
Business Unit	One bucket per BU
Cost Center	Map buckets to cost centers
Cloud Account	Separate by AWS/Azure/GCP account
Application	Critical apps get dedicated buckets

```
// Analyze log volume by bucket over last 24 hours
fetch logs, from:-1h
| summarize recordCount = count(), by:{dt.system.bucket}
| sort recordCount desc
```

Exercise 2: Recommended Naming Patterns

```
// Track log ingest trends by bucket over time
fetch logs, from:-1h
| summarize recordCount = count(), by:{time_bucket = bin(timestamp, 1h),
dt.system.bucket}
| sort time_bucket desc
```

Exercise 3: Recommended Naming Patterns

```
// Compare bucket retention configurations
fetch dt.system.buckets
| fields name, dt.system.table, retention_days
| sort retention_days desc
```

Lab Summary

You have completed 3 hands-on exercises for **ORGNZ-03: Bucket Strategy and Design**.

Exercises Completed

- ☐ Exercise 1: Recommended Naming Patterns
- ☐ Exercise 2: Recommended Naming Patterns
- ☐ Exercise 3: Recommended Naming Patterns

Next Steps

Continue with **ORGNZ-04** for the next notebook.

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.